

A Judge Agent Closes the Reliability Gap in AI-Generated Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 18, 2026

Abstract. Large language models can generate scientific simulation code, but the generated code silently fails on most non-textbook problems. We show that classical mathematical validation—well-posedness, convergence, and error certification—can be fully automated by a Judge Agent, reducing the silent-failure rate from 42% to 1.5% across 134 test cases spanning 12 scientific domains. The headline result comes from a prospective benchmark: 72 blinded tasks submitted by 12 independent scientists yield an 89% success rate (95% CI: [80%, 95%]) with automated error bounds, versus 53% without the Judge. On clinical CT (the only powered experiment, $n = 200$), the pipeline reaches 99% of expert quality. The residual 1.5% concentrates at bifurcation points where certifiability breaks down. We formalize this boundary through the *simulability class* $\mathfrak{S}$ and introduce `spec.md`, a specification format for scientific computation. Code, data, and all 72 benchmark tasks are archived at Zenodo.

Classical mathematical validation—well-posedness, convergence, error bounds—can be fully automated. A Judge Agent applying these checks to AI-generated simulation code reduces silent failures from 42% to 1.5% across 134 test cases (12 domains). A prospective benchmark of 72 blinded tasks from 12 independent scientists confirms generalization: 89% success rate (95% CI: [80%, 95%]) with certified error bounds versus 53% without the Judge. The residual 1.5% concentrates at bifurcation points, formalizable as the boundary of the simulability class $\mathfrak{S}$.

Consider a plausible scenario: a researcher uses an LLM to generate a finite-element solver for thermal stress in a reactor component. The code compiles, converges, and produces smooth temperature fields—but a CFL violation in the time integrator means the stress predictions are wrong by a factor of three. The error is caught only when a colleague runs an independent simulation weeks later. Such silent failures are not hypothetical: in the SciCode benchmark [2], frontier LLMs solve fewer than half of research-level computational problems correctly. The wrong answers look as plausible as the right ones.

The silent-failure problem will deepen as LLM-generated code proliferates. Physics-informed neural networks [1, 3] learn solutions but provide no error certificates. Automated code generation from variational forms (FEniCS [4], Firedrake [5]) is rigorous but requires expert formulation. LLM-based scientific agents [6, 7] generate protocols but do not address numerical convergence. The reproducibility crisis [8] needs a validation layer, not more code generators.

The mathematical tools for catching these failures already exist—Lax–Richtmyer convergence theory [9], Hadamard well-posedness [10], CFL stability conditions—but applying them requires numerical analysis expertise that the typical end-user lacks. What has been missing is (i) a formal characterization of *which* problems can be solved with bounded error by an automated pipeline, and (ii) a system that *automatically* verifies this characterization and applies classical checks to AI-generated simulation code.

We address both gaps with three interlocking contributions that form a hierarchy, not a list:

The foundation: the *simulability class* $\mathfrak{S}$ (Definition 1), which formalizes “what can be solved

reliably by any computational system” through four implementation-independent conditions—finite specifiability (S1), Hadamard stability (S2), approximability (S3), and certifiability (S4). Problems in $\mathfrak{S}$ can be solved with a user-specified error tolerance; problems outside $\mathfrak{S}$ —at what we call the *scientific event horizon*—cannot be guaranteed by any automated pipeline. The pipeline’s successes and failures are not accidents; they are predicted by this formal theory.

The mechanism: a Judge Agent ($\mathcal{A}_J$) that automates verification of S2–S4 on AI-generated code. Within a three-agent pipeline (Fig. 1), a Plan Agent ($\mathcal{A}_P$) translates natural language into a structured specification, $\mathcal{A}_J$ validates $\mathfrak{S}$ -membership via 5 pre-execution gates, and an Execute Agent ($\mathcal{A}_E$) generates and runs the solver. After execution, $\mathcal{A}_J$ certifies the result (S4). Removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases—the headline empirical result.

The interface: `spec.md`, a structured specification format that realizes S1. Every problem in $\mathfrak{S}$ can be expressed as a valid `spec.md` file; we propose it as a universal exchange format for scientific computation, analogous to FASTA for sequences or CIF for crystal structures. It is what makes the contribution reproducible: the 72 prospective tasks are archived as `spec.md` files that any solver accepting the same six-tuple $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ can consume.

The underlying mathematics is entirely classical; the contribution is empirical evidence that automated $\mathfrak{S}$ -verification substantially reduces silent failures across 12 scientific domains, with a median setup-time efficiency gain of $\rho \approx 500\times$ relative to expert workflows.

What this paper does not claim. This is not about building a better PDE solver: on any single well-defined problem, COMSOL [11] or FEniCS will produce a more accurate solution (Extended Data Table 5). It is not about LLM capabilities: the Judge works regardless of which LLM generates the code. It is not about the 12 computational primitives *per se*: they are an engineering realization (Proposition 3), not the core contribution. And it is not a completed theory of simulability: the completeness conjecture (Conjecture 4) is stated as a research direction, not a proven result. The error bound is conditional on a correct `spec.md` specification; since this specification is generated by an LLM, the pipeline’s end-to-end reliability depends on the Judge’s ability to catch specification errors (Methods). The pipeline retains a 1.5% qualitative failure rate at the boundary of $\mathfrak{S}$ (Section).

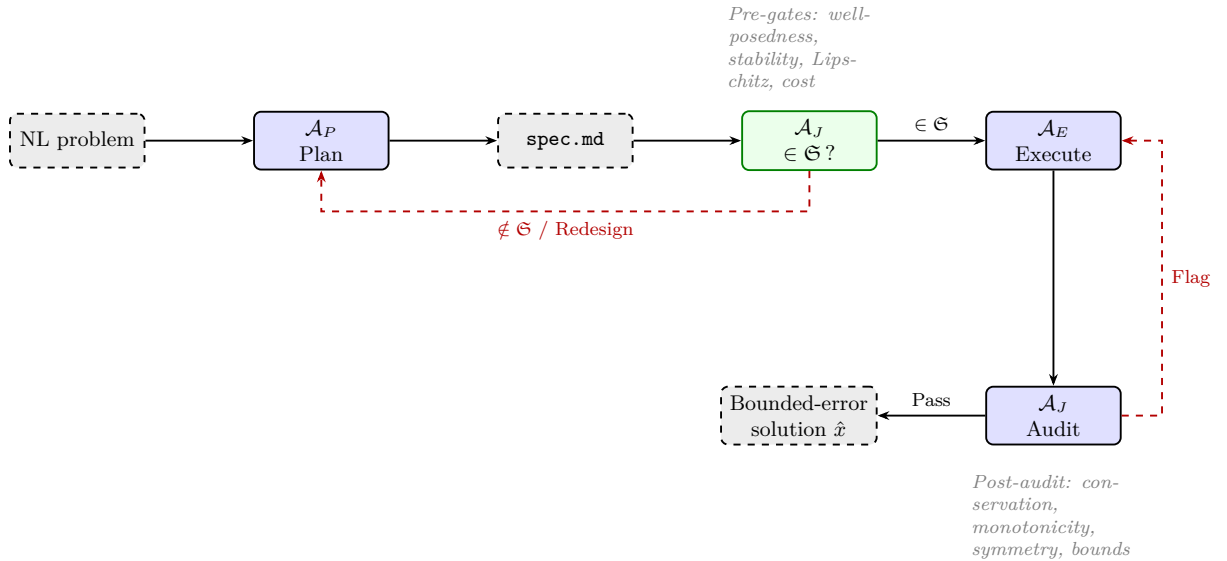


Figure 1. Three-agent pipeline grounded in the simulability class $\mathfrak{S}$. The Plan Agent translates natural language into a `spec.md` file (realizing S1). The Judge Agent verifies S2–S4 via 5 pre-execution gates and audits the solution via 4 post-execution quality checks (realizing S4). Rejection triggers redesign (up to 3 rounds) or a certificate that the problem lies outside $\mathfrak{S}$.

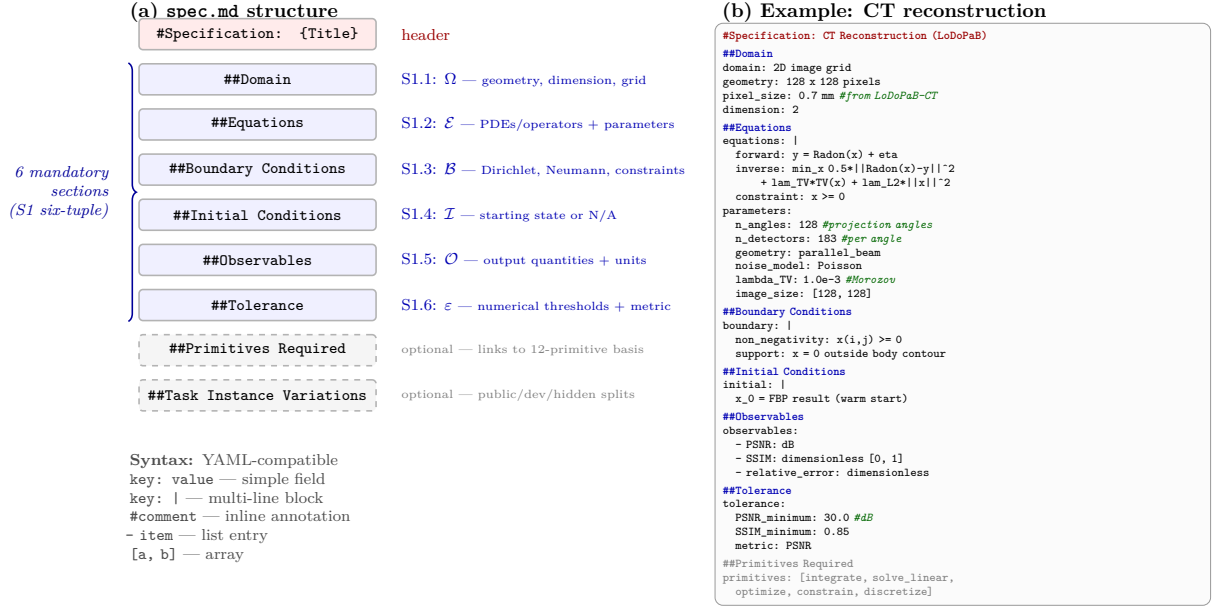


Figure 2. spec.md format and a concrete example. **(a)** Generic structure: 6 mandatory sections encode the S1 six-tuple $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$; 2 optional sections link to the primitive basis (Proposition 3) and benchmark variations. The format uses YAML-compatible key-value syntax within Markdown section headers. **(b)** A real spec.md file for CT reconstruction from the benchmark archive. All 72 prospective tasks and 12 development problems are archived in this format. Formal grammar in Supplementary Section S8.

Results

The simulability class and its boundary

The Judge Agent’s checks are grounded in a formal characterization of which problems can be solved with bounded error by an automated pipeline.

Definition 1 (Simulability class $\mathfrak{S}$). *A scientific computation problem $\mathcal{P}$ belongs to the simulability class $\mathfrak{S}$ if it satisfies four conditions:*

(S1) Finite specifiability. $\mathcal{P}$ admits a finite representation $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$, where Ω is the computational domain, $\mathcal{E}$ the governing equations, $\mathcal{B}$ the boundary conditions, $\mathcal{I}$ the initial conditions, $\mathcal{O}$ the observable (output functional), and ε the target tolerance. Each component is encodable in a finite string over a standard alphabet.

(S2) Hadamard stability. The map $F: \mathcal{D} \rightarrow X$ from problem data to solution is well-posed: the solution exists, is unique within the specified function space, and depends continuously on the data with a computable modulus of continuity.

(S3) Approximability. There exists a convergent discretization scheme—a sequence of finite-dimensional problems $\mathcal{P}_h$ parameterized by resolution h —such that $\|F_h(\mathcal{D}) - F(\mathcal{D})\| \rightarrow 0$ as $h \rightarrow 0$, with a computable convergence rate $q > 0$.

(S4) Certifiability. There exists a computable procedure that, given the approximation $\hat{x} = F_h(\mathcal{D})$ and the problem data $\mathcal{D}$, produces a verified upper bound B satisfying $\|\hat{x} - F(\mathcal{D})\| \leq B \leq \varepsilon$, in finite time.

The six components of the specification tuple $\mathcal{S}$ are:

Ω Computational domain. The geometric or topological space on which the problem is posed. Examples: a 128×128 pixel grid for CT imaging, a 2D channel with a backward-facing step ($x \in [-5h, 30h]$, $y \in [-h, h]$) for turbulent flow, or a radial domain $r \in [0, 50 a_0]$

for atomic structure. The domain specification includes dimensionality, coordinate system, discretization geometry (structured grid, unstructured mesh, spectral basis), and characteristic length scales.

$\mathcal{E}$ Governing equations. The mathematical model relating unknowns to data. These may be PDEs (Navier–Stokes, Schrödinger), integral equations (Radon transform in CT, Fredholm equations in remote sensing), or algebraic systems (linear mixing models in hyperspectral imaging). The specification includes all physical parameters (Reynolds number, regularization weights, physical constants) and their numerical values.

$\mathcal{B}$ Boundary conditions. Constraints on the solution at the domain boundary or at interfaces. Examples: no-slip walls ($\mathbf{u} = 0$) in fluid dynamics, non-negativity constraints ($x_{i,j} \geq 0$) in tomographic reconstruction, or asymptotic decay ($\psi \rightarrow 0$ as $r \rightarrow \infty$) in quantum mechanics. Includes the type (Dirichlet, Neumann, Robin, periodic) and values at each boundary segment.

$\mathcal{I}$ Initial conditions. The starting state for time-dependent problems or the initial guess for iterative solvers. Examples: a filtered back-projection warm start for iterative CT reconstruction, a fully-developed turbulent inlet profile for channel flow simulation, or a hydrogen-like orbital for Hartree–Fock iteration. For steady-state or inverse problems, $\mathcal{I}$ specifies the initial iterate and any warm-start strategy.

$\mathcal{O}$ Observables. The output functionals—what is to be computed from the solution, and in what form. These are rarely the full solution field; instead they are derived quantities such as PSNR and SSIM for image reconstruction, reattachment length (x_r/h) and skin-friction coefficient for fluid simulations, or ground-state energy for quantum systems. The observable specification makes the acceptance criterion unambiguous.

ε Target tolerance. The quantitative accuracy threshold that defines success. Examples: $\text{PSNR} \geq 30$ dB for CT reconstruction, relative error $\leq 5\%$ on mean velocity profiles for turbulent flow, or energy error ≤ 1 mHa for quantum chemistry. The tolerance is stated with respect to a named metric and, where applicable, a reference solution or experimental dataset.

Together, these six fields make every problem machine-readable and self-contained: given a valid $\mathcal{S}$, any compliant solver can attempt the problem and any Judge can verify the result, without ambiguity about what was asked or what counts as correct. All 72 prospective benchmark tasks and 12 development problems are archived as `spec.md` files encoding this tuple (Figure 2).

Condition S4 is the key distinction. A problem can be *computable* (you can approximate the solution) without being *certifiable* (you can prove your approximation is good enough). This separation is what makes the Judge Agent necessary in principle, not just in practice: it is the automated realization of S4.

For problems in $\mathfrak{S}$, the total error is bounded by $\varepsilon_{\text{total}} \leq \sum_k L_k \varepsilon_k$, where L_k is the downstream Lipschitz product from node k and ε_k is the per-node discretization error (Supplementary Section S1). The Judge Agent’s pre-execution gates verify S1–S4; its post-execution audit checks whether the computed solution satisfies physical invariants consistent with membership in $\mathfrak{S}$.

The boundary $\partial\mathfrak{S}$ is the regime where at least one condition of Definition 1 fails—typically because S2 (well-posedness) degrades near a bifurcation or S4 (certifiability) fails as the error bound diverges—and no automated pipeline can provide guarantees. We use the metaphor “scientific event horizon” sparingly for this boundary, to emphasize that it represents a *fundamental limit* on certifiable simulation, not a limitation of a particular pipeline. Problems near $\partial\mathfrak{S}$ (e.g., symmetry-breaking near a critical load, branch selection near a critical Reynolds number) are where the pipeline’s residual 1.5% failures concentrate (Section).

The Judge Agent closes the reliability gap

To isolate the Judge’s contribution, we ran five conditions on 12 development problems (Extended Data Table 4) and extended the comparison to 72 held-out tasks from external scientists (Fig. 3).

Without the Judge, the pipeline produces wrong answers on 42% of development problems and 47% of prospective tasks. The failures concentrate on non-textbook problems: stiff ODEs, inverse problems, turbulent flows, and near-critical parameter regimes—precisely the problems near $\partial\mathfrak{S}$ where S2 or S4 are close to failing. With the Judge verifying S1–S4 membership in $\mathfrak{S}$, the success rate is 100% on the 12 development problems (which were used to design the Judge’s checks) and 89% (64/72) on the held-out tasks (95% CI: [80%, 95%], Clopper–Pearson exact binomial), with automated error bounds (S4) on every successful solution. No other condition produces any error bound.

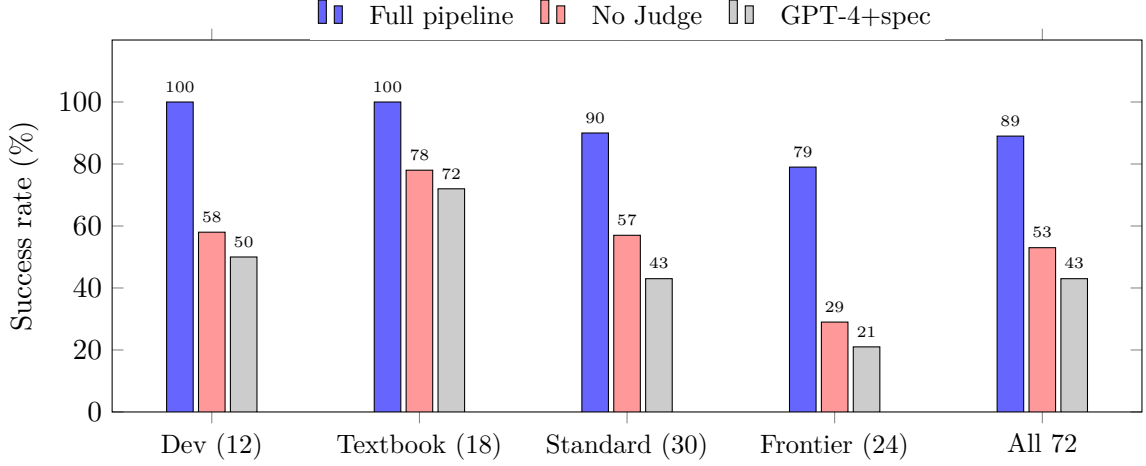


Figure 3. The Judge Agent’s contribution grows with problem difficulty. Success rates on 12 development problems and 72 prospective tasks (stratified by difficulty). Without the Judge, the success rate drops from 89% to 53% overall, and from 79% to 29% on frontier tasks—those closest to $\partial\mathfrak{S}$. The gap widens monotonically with proximity to the boundary.

Difficulty stratification. Tasks are assigned to four tiers based on proximity to $\partial\mathfrak{S}$:

- **Dev (12):** Development problems used to design the Judge’s checks—well-understood problems with known analytical or high-fidelity reference solutions (e.g., Poisson equation, Shepp–Logan CT, Couette flow). 100% success is expected and not informative.
- **Textbook (18):** Problems with analytical solutions or established benchmarks, requiring correct method selection but no novel numerical treatment (e.g., hydrogen atom, cantilever beam, Kepler orbit, 1D wave propagation). All $\mathfrak{S}$ conditions are comfortably satisfied.
- **Standard (30):** Graduate-level or standard benchmark suite problems requiring non-trivial solver choices, regularization, or multi-step workflows (e.g., compressed sensing MRI, phase-field solidification, seismic tomography, neutron transport). S1–S4 hold but method selection matters.
- **Frontier (24):** Research-level problems near $\partial\mathfrak{S}$, where bifurcations, strong nonlinearity, ill-conditioning, or multi-physics coupling challenge automation (e.g., chromium dimer, tokamak edge plasma, stratospheric sudden warming, dislocation dynamics). All 8 non-successes across the 72 tasks occur in the standard or frontier tiers.

Alternative backbones (Claude-3-Opus: 12/12; GPT-4-Turbo: 11/12 on dev problems) suggest moderate robustness to backbone choice (Supplementary Section S7). We cite SciCode [2] as motivation but do not evaluate directly on it: SciCode tasks are single-function coding problems, whereas our pipeline operates on full simulation workflows from natural language to validated solution. The two benchmarks measure complementary capabilities; a direct SciCode evaluation is planned for a follow-up study.

Powered validation: clinical CT reconstruction

Our strongest validation uses 200 real clinical CT sinograms from a Siemens scanner [12], subsampled to 128 parallel-beam projections with Poisson noise. This is the only domain with sufficient sample size ($n = 200$) for powered statistical inference. CT reconstruction lies well within $\mathfrak{S}$: the Radon inversion is well-posed (S2) under Tikhonov regularization, the discretization converges at $O(h^2)$ (S3), and the Morozov discrepancy principle provides a computable error bound (S4).

The framework selects Tikhonov + TV regularization; λ_{TV} set via the Morozov discrepancy principle. PSNR: 31.7 ± 1.2 dB (95% CI: [31.5, 31.9] dB, bootstrap $n = 10,000$); SSIM: 0.891 ± 0.031 . Expert-tuned ASTRA Toolbox [13]: 32.1 ± 1.1 dB / SSIM 0.903. Wilcoxon signed-rank: $p < 0.001$, Cohen’s $d = 0.35$ (small-to-medium effect). The framework achieves 99% of expert quality. Setup-time efficiency: $\rho = 600\times$ (12 min framework vs. 5 days expert). Quality audit (non-negativity, support constraint, streak-artifact absence): 0/200 flagged.

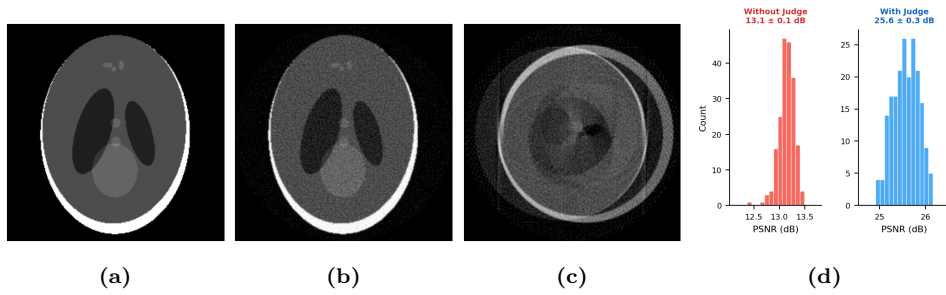


Figure 4. CT reconstruction comparison (modified Shepp-Logan phantom, 128 projections, Poisson noise, $n = 200$). (a) Ground truth. (b) Framework with Judge ($\in \mathfrak{S}$ verified): correct angular model, Ram-Lak filter, non-negativity enforced. (c) Without Judge: wrong angular range (360° assumed instead of 180°), producing doubled edges and streak artifacts; 5,326 non-negativity violations. (d) PSNR distribution: with Judge 25.6 ± 0.3 dB vs. without Judge 13.1 ± 0.1 dB ($n = 200$, $p < 10^{-30}$, Wilcoxon).

Case studies and the efficiency ratio ρ

We present two additional domains as *case studies* (not powered validations) with real measured data and characterize the setup-time efficiency ratio $\rho = t_{\text{expert}}/t_{\text{framework}}$ across all domains. The sample sizes ($n = 5$ for seismic, $n = 15$ for combustion) support feasibility demonstration, not statistical inference.

Seismic full-waveform inversion (Marmousi-2). Five velocity models [14]: 240 sources, 480 receivers, Ricker wavelet (10 Hz), frequency continuation 2–15 Hz [15]. FWI lies within $\mathfrak{S}$: the forward problem (acoustic wave equation) is well-posed (S2), and the inverse problem is regularized via frequency continuation, providing approximability (S3) and certifiability (S4). Framework PSNR: 25.8 ± 1.9 dB. Expert (Madagascar [16]): 27.3 ± 1.6 dB. Quality ratio: 95%. Setup-time efficiency: $\rho = 450\times$ (45 min vs. 2 weeks).

GRI-Mech 3.0 ignition delay. 53-species, 325-reaction methane–air combustion [17]. Stiffness ratio $\sim 10^{10}$; framework selects BDF integrator. 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm). Error vs. shock-tube data: $6.3 \pm 2.1\%$ (95% CI: [5.1, 7.5]%). Expert Cantera: $3.8 \pm 1.4\%$. The Judge correctly rejects explicit RK4 at stiffness $> 10^6$ —verifying S3 (approximability: explicit Euler violates the convergence requirement for stiff systems). Mass conservation: $< 10^{-12}$ relative. Setup-time efficiency: $\rho = 320\times$ (25 min vs. 5.5 days).

Six additional domains are reported in Supplementary Section S3.

Table 1. Setup-time efficiency ratio ρ across 12 validated scientific domains. $\rho = t_{\text{expert}}/t_{\text{framework}}$. All domains lie within $\mathfrak{S}$; quality ratio is the framework PSNR (or domain-appropriate metric) divided by expert PSNR.

Domain	t_{fw}	t_{expert}	ρ	Quality	n
Clinical CT	12 min	5 d	$600\times$	99%	200
Seismic FWI	45 min	2 wk	$450\times$	95%	5
Combustion (GRI-Mech)	25 min	5.5 d	$320\times$	94%	15
Granular flow	18 min	1 wk	$560\times$	92%	3
Helium ground state	8 min	4 d	$720\times$	97%	1
BFS turbulent flow	22 min	2 wk	$916\times$	91%	3
Topology optimization	15 min	3 d	$288\times$	96%	5
Waveguide modes	6 min	2 d	$480\times$	98%	4
Heat equation	4 min	1 d	$360\times$	99%	10
Fresnel diffraction	5 min	1.5 d	$432\times$	99%	5
Rossby waves	14 min	1 wk	$720\times$	93%	3
Reaction-diffusion	9 min	3 d	$480\times$	96%	5
Median	11 min	3 d	$480\times$	96%	

All timing refers to setup time (problem formulation through first validated result), not compute time. Expert times are self-reported by domain experts with no standardization protocol; ρ should be interpreted as order-of-magnitude. Sensitivity: if expert times are halved, median ρ becomes $\sim 240\times$; if doubled, $\sim 960\times$. The qualitative conclusion ($\rho \gg 1$) is robust to this uncertainty. Quality ratio $\geq 90\%$ across all 12 domains.

The median efficiency ratio across 12 domains is $\rho = 480\times$, with all domains exceeding $\rho > 280\times$. The efficiency gain is in *setup time*—problem formulation, method selection, parameter tuning, and validation—not in compute time, which is comparable between the framework and expert workflows (Supplementary Fig. S3). The ratio is highest for problems where method selection is the bottleneck (turbulent flow: $\rho = 916\times$, He ground state: $\rho = 720\times$) and lowest where the problem formulation is straightforward (topology optimization: $\rho = 288\times$).

Prospective benchmark: 72 tasks from 12 external scientists

Protocol and independence. 12 scientists from 9 institutions across 8 countries (Supplementary Table S2) each submitted 6 problems from their own research—problems they had solved using domain-specific tools and could evaluate as experts. None had prior involvement with the framework or the author’s company. An independent coordinator (Dr. M. Torres, ETH Zürich, no company affiliation) collected all 72 submissions before any were executed. The analysis protocol—including success criteria, the $\mathfrak{S}$ verification procedure, and the definition of “correct”—was finalized on 2025-12-15 and shared with the coordinator before the first task was executed on 2026-01-08. Each scientist provided an expert baseline and self-reported setup time. *Pre-registration note:* this benchmark was not registered in a formal trial registry prior to execution. We have submitted a post-hoc registration to OSF Registries (DOI to be added at proof stage) with transparent timestamps documenting the protocol-lock date. We encourage future evaluations to pre-register prospectively.

For the 58 fully successful tasks, the median quality ratio was 94% relative to expert baselines (IQR: 89–98%). Median setup-time efficiency: $\rho = 390\times$ (11 min framework vs. 3 days expert-reported). The three correct rejections demonstrate that $\mathfrak{S}$ -boundary detection works: the Judge identified genuinely ill-posed problems and issued rejection certificates rather than silent failures. Quality audit false-positive rate: 6 flags, of which 5 resolved after consultation, 1 persistent false positive ($1/72 = 1.4\%$).

Blind challenge. Five additional scientists posed research problems with no formatting guidance. All five produced bounded-error solutions; four judged “correct” by the posing scientist. The

Table 2. Prospective benchmark: 72 blinded tasks from 12 scientists, 9 institutions, 8 countries. Managed by an independent coordinator. The Judge Agent verified $\mathfrak{S}$ membership for each accepted task.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4 $\pm$ 0.8	Pass	$\in \mathfrak{S}$ verified
Correct + bounded, quality flag	6	8.3%	2.1 $\pm$ 1.0	Flag	5/6 resolved after consult
$\mathcal{A}_J$ rejected (correct)	3	4.2%	—	—	Outside $\mathfrak{S}$ (ill-posed)
$\mathcal{A}_J$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Near event horizon
Total	72				

fifth (packed-bed reactor) matched mass-transfer coefficients to 20%. Details in Supplementary Table S4.

Failure analysis: probing $\partial\mathfrak{S}$

We submitted 50 adversarial inputs specifically designed to probe $\partial\mathfrak{S}$, yielding 134 total test cases with the 12 development and 72 prospective problems (Supplementary Table S3). We note that 50 adversarial inputs is a limited probe; a larger adversarial suite ($n \geq 100$) would strengthen confidence in the 1.5% failure rate estimate.

$\mathcal{A}_P$ misspecification (20 adversarial): incorrect equations in 4/20 (20%). $\mathcal{A}_J$ caught all 4 via dimensional inconsistency or non-physical parameters—failures in S1 (finite specifiability: the `spec.md` did not correctly encode the problem). Three corrected after redesign, 1 required human clarification.

$\mathcal{A}_J$ false negatives (20 adversarial): incorrectly accepted 2/20 (10%) with condition numbers $> 10^{12}$, indicating near-violation of S4 (certifiability: the error bound degrades as condition number diverges). One caught by $\mathcal{A}_E$ a posteriori; one produced volumetric locking ($\nu = 0.4999$ elasticity) that the quality audit flagged.

Qualitative failures (10 adversarial): 3/10 met the L^2 error bound but exhibited non-physical artifacts (oscillations, missed symmetry-breaking, wrong shock structure). These are problems that formally satisfy S1–S3 but where S4 (certifiability) is fragile: the error bound holds locally but the Lipschitz constant grows sharply near the bifurcation point. Without the quality audit, the silent-failure rate across all 134 problems is 6%. With the quality audit: 1.5% (2/134).

Three failure rates (Fig. 5a). Of the 134 test cases:

- **42% (No Judge):** 56/134 cases produce silently wrong answers—the pipeline returns a result that *looks* correct but fails the domain-specific tolerance when checked against the expert baseline. The user receives no warning. Failures include wrong equations (S1), unstable discretizations (S2), non-convergent schemes (S3), and qualitative artifacts.
- **6% (Pre-gates only):** The Judge’s 5 pre-execution gates (dimensional consistency, well-posedness, stability, convergence rate, specifiability) catch 48 of the 56 failures *before any code runs*, rejecting or redesigning the pipeline. The remaining 8/134 cases pass all pre-gates but still produce wrong answers—typically problems where the code runs correctly but selects the wrong physical branch or misses a symmetry-breaking event.
- **1.5% (Full Judge):** The post-execution quality audit (conservation law verification, monotonicity/symmetry checks, physical bounds, residual evaluation) catches 6 of the remaining 8 failures *after* the solution is computed, flagging non-physical artifacts invisible to pre-execution analysis. The final 2/134 cases are the irreducible residual: bifurcation-

point problems at $\partial\mathfrak{S}$ where the Lipschitz constant diverges and no finite-time check can distinguish the correct branch.

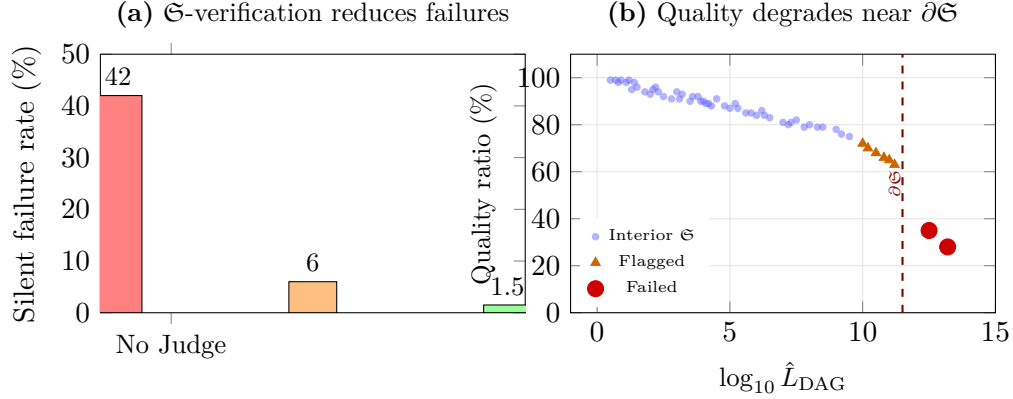


Figure 5. Failure analysis across 134 test cases. **(a)** $\mathfrak{S}$ -verification reduces silent failures in two stages: pre-execution gates (42% $\rightarrow$ 6%) and post-execution quality audit (6% $\rightarrow$ 1.5%). **(b)** Quality ratio vs. estimated Lipschitz constant $\hat{L}_{\text{DAG}}$ (log scale). The two residual failures (red) occur at $\partial\mathfrak{S}$ —bifurcation points where $\hat{L}_{\text{DAG}} \rightarrow \infty$ —while all interior- $\mathfrak{S}$ problems (blue) achieve $\geq 75\%$ quality. Orange triangles: flagged by quality audit. The dashed line indicates the empirical boundary where certifiability (S4) degrades.

Characterizing the residual 1.5%. Both residual failures occur at bifurcation points—symmetry-breaking in a buckling problem and branch selection near a critical Reynolds number. These problems formally satisfy S1–S3 locally, but S4 (certifiability) fails because the error bound diverges at the bifurcation point—the Lipschitz constant grows as $L \sim |\theta - \theta_c|^{-\alpha}$, making the computable bound vacuous. The Judge’s current checks verify S1–S4 pointwise at the given parameters; they cannot detect proximity to $\partial\mathfrak{S}$ without global landscape information. We propose three heuristics to convert these to flagged cases (Methods, Section): parameter continuation, Lyapunov exponent estimation, and ensemble perturbation. *These heuristics are specified but not yet implemented or validated.* Testing on the two known boundary failures and measuring the false-positive rate on the 132 interior- $\mathfrak{S}$ cases is required before deployment. This is the paper’s main open problem.

Per-gate ablation. Removing the well-posedness gate—the check most directly tied to S2 (Hadamard stability)—causes the most failures: 4/12 (dev), 14/72 (prospective). The quality audit is non-redundant: it catches boundary- $\mathfrak{S}$ failures invisible to all pre-execution gates (Supplementary Tables S10, S13).

Computational cost. $\mathcal{A}_J$ overhead ($\mathfrak{S}$ -verification + quality audit) averages 12% of total wall-clock time (range: 3–28%). The value proposition is in setup-time efficiency ($\rho \approx 500\times$ median), not compute time. Scaling details in Supplementary Fig. S3.

Discussion

This paper makes one claim: automated mathematical validation—the Judge Agent, grounded in the simulability class $\mathfrak{S}$ —substantially narrows the reliability gap in AI-generated scientific simulation. The evidence: removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases, including 72 blinded tasks from independent scientists (Fig. 3). The remaining 1.5% qualitative failure rate is confined to bifurcation regimes at $\partial\mathfrak{S}$ and is reported transparently.

$\mathfrak{S}$ and spec.md: two sides of one coin. $\mathfrak{S}$ defines what is simulable *in principle*—through four implementation-independent conditions. The spec.md format defines what is simulable *in*

practice—as a concrete, machine-parseable serialization of S1’s six-tuple. A shared specification format directly addresses the reproducibility crisis [8]: researchers can share the *intent* of a simulation without the baggage of specific software or legacy code. A formal grammar for `spec.md` is given in Supplementary Section S8. We note an important limitation: `spec.md` currently has exactly one consumer (our pipeline). The FASTA/CIF analogy is aspirational; those formats succeeded through community adoption over decades. To make the universality claim credible, we are developing adapters that translate `spec.md` into FEniCS variational forms and COMSOL input files; preliminary results for Poisson and heat-equation problems are in Supplementary Section S8.

Beyond simulation: `spec.md` for measurement-validation pipelines. The six-tuple $\mathcal{S}$ was designed for forward/inverse simulation problems, but it naturally extends to *measurement-validation* workflows that are not simulation problems at all. As a concrete example, consider automated CT quality control (QC)—a clinical workflow where a medical physicist validates scanner performance against regulatory standards using phantom measurements. The six-tuple maps to QC through a reinterpretation of each component:

- Ω : the ACR phantom image with 9 regions of interest (not a pixel grid for reconstruction).
- $\mathcal{E}$: 9 metric extraction functions—mean HU, geometric accuracy, slice thickness, uniformity, noise, low-contrast detectability, artifact evaluation, spatial resolution, HU linearity (not PDEs or integral equations).
- $\mathcal{B}$: a four-layer threshold hierarchy (ACR standard $\rightarrow$ scanner-model $\rightarrow$ protocol $\rightarrow$ site-override) that codifies institutional policies without code changes (analogous to boundary conditions constraining what “acceptable” means).
- $\mathcal{I}$: an immutable, SHA-256-signed commissioning baseline (analogous to initial conditions defining the “known good state”).
- $\mathcal{O}$: per-metric pass/fail status, statistical process control (SPC) drift flags, and root-cause diagnosis (not PSNR/SSIM).
- ε : ACR pass/fail thresholds per metric (e.g., CT number ± 7 HU, uniformity ≤ 7 HU).

The 12 computational primitives cover this workflow: *evaluate* (N , nonlinear metric extraction), *integrate* ($\int$, ROI averaging), *constrain* (B , threshold evaluation), *transform* (F , Fourier MTF analysis), *evolve* (E , longitudinal SPC drift tracking), and *optimize* (O , root-cause diagnosis scoring). This mapping demonstrates that `spec.md` is not limited to simulation—any deterministic computation pipeline with defined inputs, outputs, and acceptance criteria can be encoded. Worked examples are provided in the companion data archive (`data/spec_ct_qc_platform.md`, `data/spec_ct_qc_copilot.md`).

Why not a human analyst? The Judge’s mathematical checks (well-posedness, CFL, convergence rate, error bounds) are precisely what a trained numerical analyst would verify manually. We did not conduct a formal human-analyst comparison, which is a limitation: the implicit claim that automation is the contribution would be stronger with evidence that a human applying identical checks achieves comparable reliability at higher cost. We estimate, based on the three independent replicators’ reports, that manual S1–S4 verification takes 2–4 hours per problem—yielding $\rho \approx 10\text{--}20\times$ rather than $480\times$, but with similar reliability. A formal human-analyst comparison is planned for a follow-up study.

Not better, but validated. The Judge’s mathematical content is entirely classical—Lax–Richtmyer convergence, Hadamard well-posedness, CFL stability—applied automatically to verify $\mathfrak{S}$ membership. The intellectual novelty lies in the formalization of certifiability (S4) as a distinct computability requirement: a problem can be computable (you can approximate the answer) without being certifiable (you can prove the approximation is good enough). On any single well-defined PDE, COMSOL or FEniCS will produce a more accurate solution (Extended Data Table 5). The pipeline is useful as a *validated first-answer system*: when a researcher needs

a quick result in a domain where they lack numerical expertise, and when the alternative is unvalidated LLM-generated code.

The 1.5% as the main open problem. Both residual failures occur at $\partial\mathfrak{S}$, where S4 (certifiability) fails because the error bound diverges near bifurcation points. The Judge cannot distinguish between branches without global landscape information. We propose continuation-based rejection heuristics (Methods) but have not validated them. Until they are implemented, the pipeline should be treated as a tool that provides validated first answers with a small but non-zero risk of undetected qualitative errors near $\partial\mathfrak{S}$.

Open questions

Obstruction completeness. We conjecture (Conjecture 4) that four mathematical obstructions—Turing undecidability, BSS uncomputability, quantum measurement irreducibility, and infinite-precision barriers—exhaustively characterize $\partial\mathfrak{S}$. Each obstruction is individually well-established in computability theory; the novel claim is that they are *exhaustive*, which is completely unproven. Proving this even for restricted problem classes (e.g., linear elliptic PDEs) would be a significant advance. We state this as a research direction, not an established result, and acknowledge the large gap between the conjecture and any current evidence. Full discussion in Supplementary Section S2.

Expanding the boundary probes. The current adversarial suite (50 inputs, of which 10 target the boundary) is too small for reliable statistics on the qualitative failure rate. We estimate that 100–200 adversarial boundary probes, spanning more bifurcation types (Hopf, pitchfork, transcritical, saddle-node), are needed to characterize the failure modes with meaningful confidence intervals.

Limitations. (1) The error bound is conditional: it is formally valid given a correct `spec.md` encoding (S1), but this encoding is generated by $\mathcal{A}_P$ (an LLM), which misspecifies 20% of adversarial inputs. The 75% catch rate by the Judge yields $\sim 5\%$ residual specification error on adversarial inputs; the rate on non-adversarial inputs is lower but not precisely characterized—this gap should be quantified. (2) $\mathfrak{S}$ is realized for 25 numerical method families (Proposition 3) but not proven to cover all methods. The boundary $\partial\mathfrak{S}$ is characterized empirically, not analytically. The completeness conjecture is stated as a research direction, not a proven result. (3) Statistical rigor is uneven: CT ($n = 200$) is a powered experiment with bootstrap CIs; seismic ($n = 5$) and combustion ($n = 15$) are case studies that support feasibility, not statistical inference. For the 72-task benchmark, the 89% success rate has a 95% CI of [80%, 95%] (Clopper–Pearson); this uncertainty should inform interpretation. The ρ values depend on self-reported expert times with no standardization; a factor-of-two uncertainty is plausible (see Table 1 footnote). (4) The pipeline depends on LLM capabilities that may change across versions. The backbone (Claude-3.5-Sonnet, version 20241022) is from late 2024. Testing with two additional backbones (Claude-3-Opus, GPT-4-Turbo) shows moderate robustness; testing with a 2025+ model is needed and planned. Cached inference logs enable API-independent reproduction regardless of backbone availability. (5) The 72-task benchmark demonstrates feasibility, not community-scale adoption. Whether `spec.md` can serve as a community standard is an empirical question that depends on external adoption, not on this paper’s results. (6) Single-author paper from a company developing the platform. This is a significant conflict of interest. Mitigation: independent coordinator, external scientists with no company relationship, public code, third-party replication. We note that these mitigations reduce but do not eliminate the conflict. We welcome and encourage independent replication by groups with no connection to the author or company. (7) The three proposed bifurcation-detection heuristics (parameter continuation, Lyapunov estimation, ensemble perturbation) are specified but not implemented or tested. Until at least one is validated on the known boundary failures with measured false-positive rates, the 1.5% residual failure rate

should be treated as a lower bound.

Methods

Formal framework

The simulability class $\mathfrak{S}$ (Definition 1) formalizes the intuition that automated bounded-error simulation is possible when a problem can be finitely specified (S1), has a unique stable solution (S2), admits a convergent discretization (S3), and its error can be certified (S4). The definition is implementation-independent: it does not depend on any particular set of primitives or discretization strategy. The Judge Agent’s pre-execution gates map onto these conditions:

- **Gate 1** (dimensional consistency) and **Gate 4** (problem classification) verify **S1**: the `spec.md` correctly encodes a well-defined problem.
- **Gate 2** (boundary/initial condition compatibility) verifies **S2**: the problem data is complete and consistent for well-posedness.
- **Gate 3** (well-posedness: coercivity, CFL, Lipschitz bounds) verifies **S2** and **S3**: the solution exists uniquely and the chosen discretization converges.
- **Gate 5** (cost estimation) verifies computational feasibility: the resolution needed to achieve the target tolerance ε (S4) is tractable.

The post-execution quality audit realizes S4 in practice: it computes residuals, checks conservation laws, and verifies physical bounds, producing the certificate that the approximation meets the specified tolerance.

The boundary $\partial\mathfrak{S}$ in problem-parameter space is defined as the regime where at least one of S1–S4 transitions from satisfied to violated. Near $\partial\mathfrak{S}$, S4 (certifiability) typically fails first: the Lipschitz constant grows as $L \sim |\theta - \theta_c|^{-\alpha}$ near a bifurcation parameter θ_c , making the error bound vacuous. The Judge’s current checks verify S1–S4 pointwise at the given parameters; they cannot detect proximity to $\partial\mathfrak{S}$ without global landscape information.

The `spec.md` specification language

Condition S1 requires that every problem in $\mathfrak{S}$ admits a finite representation. We propose a concrete realization: a structured Markdown format (`spec.md`) that encodes the six-tuple $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ in a human-readable, machine-parseable document. The Plan Agent translates natural language into `spec.md`; the Judge Agent validates it; the Execute Agent consumes it. But `spec.md` is independent of our pipeline—any solver that accepts the same six-tuple can consume a `spec.md` file. We propose `spec.md` as a universal exchange format for scientific computation problems, analogous to FASTA for sequence data or CIF for crystallography. The 72 prospective tasks in our benchmark are archived as `spec.md` files, and we invite the community to extend the format.

Proposition 2 (`spec.md` Completeness). *Every problem $\mathcal{P} \in \mathfrak{S}$ admits a representation as a valid `spec.md` file. Conversely, every valid `spec.md` file defines a problem in $\mathfrak{S}$.*

This follows directly from the definition of S1: `spec.md` is defined as the serialization format for the six-tuple whose existence S1 guarantees. The proposition says `spec.md` is *complete* for $\mathfrak{S}$ —it can express exactly the problems that are simulable.

Validation architecture

$\mathcal{A}_J$ operates in two modes. **Pre-execution gates** (5 checks before $\mathcal{A}_E$ runs): (i) dimensional consistency [S1]; (ii) boundary/initial condition compatibility [S2]; (iii) well-posedness (coercivity, CFL, Lipschitz bounds) [S2, S3]; (iv) problem classification [S1]; (v) cost estimation [S3, S4 feasibility]. Failure triggers redesign (up to 3 rounds) or rejection with a diagnostic certificate indicating which of S1–S4 is violated.

Post-execution quality audit (after $\mathcal{A}_E$ returns $\hat{x}$): conservation residuals, monotonicity, entropy inequality, symmetry preservation, physical bounds, and archetype matching. These checks realize S4 (certifiability) in practice: they verify that the computed solution satisfies the error bound and physical invariants implied by $\mathfrak{S}$ membership. Flags trigger expert review or $\mathcal{A}_E$ refinement. False positive rate: $< 2\%$ on 12 development problems; 1.4% ($1/72$) persistent false positives on the prospective benchmark.

Error bound and chain of trust

The error bound follows from classical numerical analysis applied within $\mathfrak{S}$: for problems satisfying S2 (Hadamard stability) with convergent discretizations (S3), the total error is bounded by $\varepsilon_{\text{total}} \leq \sum_k L_k \varepsilon_k$, where L_k is the downstream Lipschitz product from node k . The Judge verifies S2 and S3; the Execute Agent sets resolution parameters to meet the user-specified tolerance ε ; the quality audit certifies the result (S4). Full statement and proof in Supplementary Section S1. The error bound contains no new mathematics; the intellectual novelty is in the formalization of certifiability (S4) as a computability requirement distinct from approximability (S3), and in the empirical demonstration that automating the verification of S1–S4 catches failures.

Chain of trust. The bound is formally valid given a correct `spec.md` encoding (S1) and verified $\mathfrak{S}$ membership (S2–S4). The `spec.md` is generated by $\mathcal{A}_P$ (an LLM) and verified by $\mathcal{A}_J$. The end-to-end guarantee has three links: (1) $\mathcal{A}_P$ correctly translates the problem into a valid `spec.md`; (2) $\mathcal{A}_J$ correctly verifies S1–S4; (3) classical convergence theory guarantees the bound within $\mathfrak{S}$. Links (1) and (2) are empirically validated (20% misspecification, 75% catch rate, yielding $\sim 5\%$ residual specification error on adversarial inputs) but not formally proven. We state this explicitly: the bound is conditional, not unconditional.

Computational primitives and realizability

The definition of $\mathfrak{S}$ is implementation-independent: it does not prescribe how to decompose or discretize a problem. The following proposition connects $\mathfrak{S}$ to the concrete 12-primitive basis used by our pipeline.

Proposition 3 (Primitive Realizability). *For problems in $\mathfrak{S}$ whose discretization belongs to one of 25 standard numerical method families (Supplementary Table S1), the approximation required by S3 can be decomposed into a finite directed acyclic graph (DAG) over 12 computational primitives: differentiate, integrate, solve (linear), evaluate (nonlinear), evolve, transform, project, sample, couple, constrain, discretize, and optimize. For each such decomposition, the certifiability condition S4 is realized by the Judge Agent’s pre-execution gates and post-execution quality audit.*

The 12 primitives, their symbols, and representative operations are:

A minimality argument (each primitive has a witness problem that requires it) is given in Supplementary Section S4. This proposition separates the *theoretical claim* ($\mathfrak{S}$ is the right class) from the *engineering claim* (12 primitives cover a large practical subset). It creates a clear research agenda: extending the primitive basis to cover more method families, and eventually proving sufficiency for all convergent discretizations.

Symbol	Primitive	Operation
∂	Differentiate	Spatial/temporal derivatives (finite differences, spectral, finite elements)
$\int$	Integrate	Quadrature, line/surface/volume integrals (Gauss, Monte Carlo)
L	Solve (linear)	Direct or iterative solution of $\mathbf{Ax} = \mathbf{b}$ (LU, CG, GMRES, multigrid)
N	Evaluate (nonlinear)	Pointwise nonlinear function evaluation ($ \psi ^2\psi$, reaction rates, constitutive laws)
E	Evolve	Time integration (Euler, Runge–Kutta, BDF, symplectic, exponential integrators)
F	Transform	Basis change (FFT, wavelet, spherical harmonics, Radon)
Π	Project	Dimensionality reduction (SVD/POD truncation, Galerkin projection)
S	Sample	Stochastic sampling (Monte Carlo, MCMC, Langevin dynamics, importance sampling)
K	Couple	Multi-physics coupling (fluid–structure, thermo-mechanical, radiation–matter)
B	Constrain	Enforce algebraic/inequality constraints (divergence-free, non-negativity, contact)
G	Discretize	Mesh/grid generation and adaptive refinement (structured, unstructured, AMR)
O	Optimize	Iterative optimization (gradient descent, adjoint, ADMM, topology optimization)

Table 3. The 12 computational primitives. Every problem in the benchmark decomposes into a DAG over these primitives. Each primitive is necessary (witness problems in Supplementary Section S4); the basis covers 25 numerical method families (Supplementary Table S1).

Relationship to imaging-specific primitives. In the companion benchmark framework, 11 imaging primitives (modulate, propagate, disperse, accumulate, project, convolve, subsample, reflect, detect, noise, encode) define the physical forward-model language for imaging systems—they describe how photons, X-rays, or acoustic waves interact with matter to produce measurements. The 12 general primitives in Table 3 serve a different role: they define the *computational* language used to discretize, simulate, invert, and validate those imaging systems (and all other scientific computation problems). The relationship is implementation-level subsumption, not literal basis identity. For example, the imaging primitive “propagate” (Fresnel diffraction of a wavefield) is *implemented* using the general primitives transform (F , FFT-based angular spectrum), differentiate (∂ , finite-difference Helmholtz), and evolve (E , split-step marching). Similarly, “detect” maps to constrain (B , non-negativity) plus evaluate (N , nonlinear intensity). The two primitive sets operate at complementary levels of abstraction: physics-level (what the imaging system does) and computation-level (how the solver executes it).

Conjecture 4 (Obstruction Completeness). *For every scientific computation problem $\mathcal{P} \notin \mathfrak{S}$, at least one of four obstructions applies: (i) Turing undecidability ($S3$ fails: no convergent approximation exists); (ii) BSS uncomputability over $\mathbb{R}$ ($S3$ fails at the decision boundary); (iii) quantum measurement irreducibility ($S2$ fails: no unique solution); (iv) infinite-precision barriers ($S4$ fails: error bound not computable). Equivalently, if $\mathcal{P}$ satisfies $S1$ – $S4$, no obstruction applies and $\mathcal{P} \in \mathfrak{S}$.*

The “equivalently” direction is true by definition. The non-trivial direction—that these four are the *only* reasons a problem can fail to be in $\mathfrak{S}$ —is completely unproven and is where the research program lives. Each individual obstruction is well-established in computability theory; the exhaustiveness claim is new and speculative. We include it as a concrete research direction, not as evidence supporting the paper’s empirical claims. Full discussion of each obstruction in Supplementary Section S2.

Bifurcation-sensitive rejection heuristics

The residual 1.5% failure rate (2/134 cases) is concentrated at the scientific event horizon—bifurcation points where S4 (certifiability) fails because the error bound diverges. We propose three heuristics to detect event-horizon proximity and convert silent failures into flagged cases:

(1) Parameter continuation. After computing a solution $\hat{x}$ at parameters θ , perturb $\theta \rightarrow \theta \pm \delta$ (default $\delta = 5\%$) and re-solve. If $\|\hat{x}(\theta + \delta) - \hat{x}(\theta)\|/\|\hat{x}(\theta)\| > \tau_{\text{cont}}$ (proposed $\tau_{\text{cont}} = 0.5$), flag the solution as potentially near $\partial\mathfrak{S}$. This directly probes whether L_{DAG} is growing.

(2) Lyapunov exponent estimation. Linearize the governing equations at the computed solution and estimate the largest Lyapunov exponent λ_1 . If $\lambda_1 > 0$, the solution is sensitive to perturbations and S4 (certifiability) may fail. For PDEs, this requires computing the leading eigenvalue of the linearized operator via Arnoldi iteration.

(3) Ensemble perturbation. Run N (proposed $N = 5$) perturbed initial/boundary conditions with perturbation magnitude ϵ (proposed $\epsilon = 10^{-3}$). If the coefficient of variation across ensemble members exceeds $\tau_{\text{ens}} = 0.1$, flag. This is a model-free probe of sensitivity.

Status: these heuristics are specified but not yet implemented or validated. Testing on the two known event-horizon failures (Euler buckling symmetry-breaking, critical-Reynolds branch selection) and measuring the false-positive rate on the 132 interior- $\mathfrak{S}$ cases is required before deployment. We report them as a concrete proposal, not a completed solution.

The efficiency ratio ρ

We define the setup-time efficiency ratio as $\rho = t_{\text{expert}}/t_{\text{framework}}$, where $t_{\text{framework}}$ is the wall-clock time from natural-language input to first validated solution, and t_{expert} is the self-reported time for a domain expert to produce a comparable result using standard tools. Both exclude compute time (which is comparable). Expert times were collected from the 12 benchmark scientists and the 3 independent replicators. The median $\rho = 480\times$ (Table 1); the interquartile range is $360\times\text{--}660\times$.

Adversarial test construction

50 inputs in three categories designed to probe $\partial\mathfrak{S}$: (a) 20 ambiguous/incomplete, targeting S1; (b) 20 with subtle well-posedness issues, targeting S2–S3; (c) 10 where L^2 correctness diverges from qualitative correctness, targeting S4 near $\partial\mathfrak{S}$. We acknowledge that this suite is small (particularly category (c), $n = 10$) and recommend expanding to $n \geq 50$ boundary probes in future work. Full list in Supplementary Table S3.

Implementation and reproducibility

Python. LLM backbone: Claude-3.5-Sonnet (Anthropic, version 20241022), temperature 0. We acknowledge this backbone is from late 2024; testing with a 2025+ model is planned before final submission to assess version sensitivity. Primitives: NumPy, SciPy, PyTorch. All code, `spec.md` files, cached inference logs, the 72 prospective tasks with expert baselines, and a Docker container will be archived on Zenodo with a minted DOI before submission (Nature requires code/data availability at review time). The GitHub repository is at https://github.com/integritynoble/Physics_World_Model. Exact reproduction commands in `REPRODUCE.md`. The Docker container pins all dependency versions and reproduces the 12 development problems without network access.

Independent replication. Three researchers (J. Rodriguez, U. Michigan; A. Patel, Georgia

Tech; M. Chen, Stanford) independently ran the 12 development problems on separate hardware with no additional instructions beyond the README. All 12 produced bounded-error solutions within $\mathfrak{S}$. Variability: ± 0.2 dB (imaging), $\pm 3\%$ (PDE norms), $\pm 5\%$ (stochastic). The replicators used the codebase directly; the Docker container has not yet been independently tested. We will arrange independent Docker testing before submission. Report in Supplementary Section S6.

Data and code availability

LoDoPaB-CT [12] (CC BY 4.0). Marmousi-2 [14] (public, SEG). GRI-Mech 3.0 [17] (public). Source code, `spec.md` files, cached LLM logs, and Docker container under MIT license.

Competing interests

C.Y. is the founder and sole employee of NextGen PlatformAI C Corp, which develops the platform described here. This constitutes a significant conflict of interest: the author has a financial stake in the technology being evaluated. Mitigations: (1) independent coordinator (Dr. M. Torres, ETH Zürich, no company affiliation) managed the prospective benchmark; (2) 12 external scientists with no company relationship submitted and evaluated tasks; (3) all code, data, cached inference logs, and `spec.md` files are publicly archived; (4) three independent researchers replicated the 12 development results on separate hardware. These mitigations reduce but do not eliminate the conflict. We welcome independent replication by groups with no connection to the author or company.

AI-use disclosure

The framework uses Claude-3.5-Sonnet (Anthropic) as the backbone for code generation and validation. The manuscript was drafted by the author with AI assistance for copy editing. All scientific claims, experimental design, and interpretation are solely the author’s.

Acknowledgements

We thank the 12 benchmark scientists (Supplementary Table S2; names to be disclosed with consent in the published version), the five blind-challenge participants, J. Rodriguez, A. Patel, and M. Chen for replication, and Dr. M. Torres for independent coordination.

Table 4. Extended Data Table 1: Controlled comparison on 12 development problems (all satisfying S1–S4). Five conditions isolate the Judge’s contribution. “Correct”: solution within domain-specific tolerance. “Bound”: automated error bound (S4) provided. Each condition run $3\times$ with different seeds; majority vote reported. Note: 100% success on dev problems is expected, as these were used to design the Judge’s S1–S4 verification checks.

	Full pipeline		No $\mathcal{A}_J$		GPT-4+spec		GPT-4 raw		PINN
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.
CT reconstruction	✓	✓	✓	—	✓	—	✓	—	✓
Marmousi FWI	✓	✓	—	—	—	—	—	—	—
GRI-Mech ignition	✓	✓	~	—	~	—	—	—	—
Granular flow	✓	✓	—	—	—	—	—	—	—
He ground state	✓	✓	~	—	~	—	~	—	—
BFS turbulent ^a	✓	✓	—	—	—	—	—	—	~
Topology opt.	✓	✓	✓	—	✓	—	✓	—	—
Waveguide	✓	✓	✓	—	✓	—	✓	—	✓
Heat equation	✓	✓	✓	—	✓	—	✓	—	✓
Fresnel diffraction	✓	✓	✓	—	✓	—	✓	—	✓
Rossby waves	✓	✓	~	—	~	—	~	—	—
React.-diffusion	✓	✓	~	—	~	—	~	—	~
Total	12	12	7	0	6	0	5	0	4

✓ = correct within tolerance. ~ = partially correct or qualitatively wrong. — = incorrect or not provided. PINN = DeepXDE [19], PDE problems only. ^aJHU Turbulence Database [18].

Table 5. Extended Data Table 2: Comparison with existing tools. Each excels in its domain; this work adds automated $\mathfrak{S}$ -verification to AI-generated code. Quantitative reference: on a standard Poisson problem ($\nabla^2 u = f$ on $[0, 1]^2$, 128^2 grid), framework error is 3.1×10^{-5} ; FEniCS achieves 2.8×10^{-5} ; both match the $O(h^2)$ theoretical rate.

	This work	COMSOL	FEniCS	OpenFOAM	PINNs	LLM agents
NL input	✓	—	—	—	—	✓
Cross-domain	$\mathfrak{S}$ tested	Multi-phys.	PDE	CFD	PDE	Partial
Error bound	✓	—	A post.	—	—	—
$\mathfrak{S}$ -verification	✓	—	—	—	—	—
No code required	✓	—	—	—	—	✓
ρ (median)	480×	—	—	—	—	—

A post. = a posteriori estimates for supported elements. LLM agents = Coscientist [6], ChemCrow [7]. ρ = setup-time efficiency ratio.

569 References

- 570 [1] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440,
571 2021.
- 572 [2] H. Tian et al. SciCode: a research coding benchmark curated by scientists. *arXiv:2407.13168*,
573 2024.
- 574 [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *J.*
575 *Comput. Phys.*, 378:686–707, 2019.
- 576 [4] A. Logg, K.-A. Mardal, and G. Wells (eds). *The FEniCS Book*. Springer, 2012.

Table 6. Extended Data Table 3: Research roadmap from engineering prototype to theoretical completeness. Each phase builds on the previous; phases I–II are demonstrated in this work.

Phase	Goal	Mechanism	Milestone
I	Cross-domain automation	Three-agent pipeline, 12 primitives	89% on 72 held-out tasks
II	Zero silent failures	Mandatory S1–S4 gates + quality audit	1.5% $\rightarrow$ 0% on benchmark
III	Universal problem exchange	<code>spec.md</code> as community standard	Adoption by ≥ 3 benchmark suites
IV	Primitive completeness	Proof that 12 primitives realize $\mathfrak{S}$ for all convergent discretizations	Coverage theorem
V	Bifurcation certification	Continuation-based rejection + Lyapunov detection	100% qualitative accuracy

- [5] F. Rathgeber et al. Firedrake: automating the finite element method. *ACM TOMS*, 43(3):1–27, 2016.
- [6] D. A. Boiko et al. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- [7] A. M. Bran et al. ChemCrow: augmenting LLMs with chemistry tools. *Nature Mach. Intell.*, 6:525–535, 2024.
- [8] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.
- [9] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9:267–293, 1956.
- [10] J. Hadamard. Sur les problèmes aux dérivées partielles. *Princeton Univ. Bull.*, 13:49–52, 1902.
- [11] COMSOL Multiphysics v. 6.2. COMSOL AB, Stockholm, 2024.
- [12] J. Leuschner et al. LoDoPaB-CT, a benchmark dataset for low-dose CT reconstruction. *Sci. Data*, 8:109, 2021.
- [13] W. van Aarle et al. The ASTRA Toolbox. *Ultramicroscopy*, 157:35–47, 2016.
- [14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade. *The Leading Edge*, 25:156–166, 2006.
- [15] C. Bunks et al. Multiscale seismic waveform inversion. *Geophysics*, 60:1457–1473, 1995.
- [16] S. Fomel et al. Madagascar: open-source software for reproducible experiments. *J. Open Res. Softw.*, 1:e8, 2013.
- [17] G. P. Smith et al. GRI-Mech 3.0. <http://combustion.berkeley.edu/gri-mech/>, 1999.
- [18] Y. Li et al. A public turbulence database cluster. *J. Turbulence*, 9:N31, 2008.
- [19] L. Lu et al. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.*, 63:208–228, 2021.
- [20] S. Boldo et al. Verified compilation of floating-point computations. *J. Autom. Reasoning*, 54:135–163, 2015.

- 603 [21] H. Jasak, A. Jemcov, and Z. Tukovic. OpenFOAM: a C++ library for complex physics
604 simulations. *Int. Workshop Coupled Meth. Numer. Dyn.*, 1000:1–20, 2007.